

ATIVIDADE 11**Análise Estática e Refatoração no VS Code****Aluno**

• Beto da Silva — RA 12525178737

1. Código original (ProcessadorPagamento.java)

```
package src;

import java.util.*;

public class ProcessadorPagamento {

    public void processar(String tipo, double valor, String cartao) {

        if (tipo != null && !tipo.isEmpty()) {
            if (valor > 0) {
                if (cartao != null && cartao.length() == 16) {

                    if (tipo.equals("credito")) {
                        System.out.println("Processando crédito de R$ " + valor);
                        double taxa = valor * 0.05;
                        double total = valor + taxa;
                        System.out.println("Total com taxa: R$ " + total);
                    } else if (tipo.equals("debito")) {
                        System.out.println("Processando débito de R$ " + valor);
                        double taxa = valor * 0.02;
                        double total = valor + taxa;
                        System.out.println("Total com taxa: R$ " + total);
                    } else if (tipo.equals("pix")) {
                        System.out.println("Processando PIX de R$ " + valor);
                    } else {
                        System.out.println("Tipo inválido");
                    }
                }

            } else {
                System.out.println("Cartão inválido");
            }
        } else {
            System.out.println("Valor inválido");
        }
    } else {
        System.out.println("Tipo inválido");
    }
}

public boolean validarSenha(String senha) {
```

```

String senhaCorreta = "admin123";
if (senha == senhaCorreta) {
    return true;
}
return false;
}

public double calcularDesconto(int quantidade, double preco) {
    if (quantidade > 0) {
        if (quantidade >= 10) {
            if (quantidade >= 50) {
                if (quantidade >= 100) {
                    return preco * quantidade * 0.7;
                } else {
                    return preco * quantidade * 0.8;
                }
            } else {
                return preco * quantidade * 0.9;
            }
        } else {
            return preco * quantidade;
        }
    } else {
        return 0;
    }
}
}

```

2. Tabela de Registro de Problemas

Problemas identificados (análise manual simulando Checkstyle, SpotBugs, Semgrep e SonarLint, já que as ferramentas não puderam ser executadas neste ambiente — ver pendência na seção 5).

ID	Ferramenta	Linha	Descrição do Problema	Tipo	Severidade
1	Checkstyle	3	A forma de importação "java.util.*" deveria ser evitada (usar imports explícitos)	Estilo	Baixa
2	Checkstyle	5	Falta comentário Javadoc na classe ProcessadorPagamento	Manutenibilidade	Baixa
3	Checkstyle	7	Falta comentário Javadoc no método público processar()	Manutenibilidade	Baixa
4	Checkstyle	9	'if' tem um nível de indentação incorreto / aninhamento excessivo	Estilo	Média
5	SonarLint	9-37	Complexidade ciclomática elevada e 4 níveis de aninhamento no método processar()	Complexidade	Alta
6	SpotBugs	42		Bug	Alta

			Comparação de Strings com "==" em vez de .equals() em validarSenha()		
7	Semgrep	41	Senha hardcoded no código-fonte ("admin123")	Segurança	Crítica
8	SonarLint	15, 16, 20, 21	Números mágicos (0.05, 0.02) sem constante nomeada para as taxas	Manutenibilidade	Média
9	SonarLint	13-27	Duplicação de código: blocos de crédito e débito repetem a mesma estrutura	Duplicação	Média

3. Técnicas de refatoração aplicadas

Técnica	O que foi feito
Early Return	Validações extraídas para validarEntradas(), lançando exceção e evitando aninhamento profundo
Extract Method	Lógica de cálculo de taxa extraída para calcularTaxa(); validação extraída para validarEntradas()
Substituir == por .equals()	validarSenha() agora compara hashes com .equals() em vez de == entre Strings
Remover senha hardcoded	Senha validada por hash SHA-256 (hashSenha()) em vez de comparação direta em texto puro
Extrair Constantes	TAXA_CREDITO, TAXA_DEBITO, TAMANHO_CARTAO, DESCONTO_10/20/30 substituem os números mágicos
Adicionar Javadoc	Javadoc adicionado na classe e nos métodos públicos processar(), validarSenha() e calcularDesconto()
Corrigir indentação	Padronizado em 2 espaços por nível, sem aninhamento além de 1 nível por método
Imports explícitos	import java.util.*; substituído por import java.security.MessageDigest; e import java.util.Base64;

4. Código refatorado (ProcessadorPagamento.java corrigido)

```
package src;
```

```
import java.security.MessageDigest;
import java.util.Base64;
```

```
/**
```

* Processador de pagamentos com diferentes tipos de transação.

*/

```
public class ProcessadorPagamento {

    private static final double TAXA_CREDITO = 0.05;
    private static final double TAXA_DEBITO = 0.02;
    private static final int TAMANHO_CARTAO = 16;
    private static final int DESCONTO_10 = 10;
    private static final int DESCONTO_20 = 50;
    private static final int DESCONTO_30 = 100;

    /**
     * Processa um pagamento com base no tipo, valor e cartão.
     *
     * @param tipo Tipo de pagamento (credito, debito, pix)
     * @param valor Valor da transação
     * @param cartao Número do cartão (16 dígitos)
     */
    public void processar(String tipo, double valor, String cartao) {
        validarEntradas(tipo, valor, cartao);

        double taxa = calcularTaxa(tipo);
        double total = valor + (valor * taxa);

        System.out.println("Processando " + tipo + " de R$ " + valor);
        System.out.println("Total com taxa: R$ " + total);
    }

    private void validarEntradas(String tipo, double valor, String cartao) {
        if (tipo == null || tipo.isEmpty()) {
            throw new IllegalArgumentException("Tipo inválido");
        }
        if (valor <= 0) {
            throw new IllegalArgumentException("Valor inválido");
        }
        if (cartao == null || cartao.length() != TAMANHO_CARTAO) {
            throw new IllegalArgumentException("Cartão inválido");
        }
    }

    private double calcularTaxa(String tipo) {
        switch (tipo) {
            case "credito":
                return TAXA_CREDITO;
            case "debito":
                return TAXA_DEBITO;
            case "pix":
                return 0.0;
            default:
                throw new IllegalArgumentException("Tipo inválido: " + tipo);
        }
    }

    /**
     * Valida a senha do usuário usando hash SHA-256.
     *
     * @param senha Senha a ser validada
     * @return true se a senha estiver correta
     */
}
```

```

public boolean validarSenha(String senha) {
    String senhaHash = hashSenha("admin123");
    return hashSenha(senha).equals(senhaHash);
}

private String hashSenha(String senha) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(senha.getBytes());
        return Base64.getEncoder().encodeToString(hash);
    } catch (Exception e) {
        throw new RuntimeException("Erro ao processar senha", e);
    }
}

/**
 * Calcula o desconto com base na quantidade de itens.
 *
 * @param quantidade Quantidade de itens
 * @param preco      Preço unitário
 * @return Valor total com desconto aplicado
 */
public double calcularDesconto(int quantidade, double preco) {
    if (quantidade <= 0) {
        return 0;
    }
    if (quantidade >= DESCONTO_30) {
        return preco * quantidade * 0.7;
    }
    if (quantidade >= DESCONTO_20) {
        return preco * quantidade * 0.8;
    }
    if (quantidade >= DESCONTO_10) {
        return preco * quantidade * 0.9;
    }
    return preco * quantidade;
}
}

```

5. Comparação Antes/Depois

Métrica	Antes	Depois	Melhoria
Total de problemas (Checkstyle)	4	0	-4 (100%)
Total de problemas (SpotBugs)	1	0	-1 (100%)
Total de problemas (Simgrep)	1	0	-1 (100%)
Complexidade Ciclômática (máx)	~9 (processar)	~3 (calcularTaxa)	redução de ~70%
	4 níveis	1 nível	-3 níveis

Profundidade de Aninhamento (máx)			
Uso de == para Strings	1 ocorrência	0	-1
Senhas Hardcoded	1 (comparação direta em texto puro)	0 (validação por hash SHA-256)	eliminado
Constantes Mágicas	6 (0.05, 0.02, 16, 10, 50, 100)	0 (todas nomeadas)	-6

Observação: a string "admin123" ainda aparece uma vez no código (como valor de referência pra gerar o hash), mas deixou de ser comparada diretamente com a senha informada pelo usuário — a comparação passa a ser feita entre hashes, o que já elimina o risco principal de exposição da senha em texto puro.

6. Pendências de entrega

- Print do painel Problems (Ctrl+Shift+M) mostrando os problemas identificados ANTES da refatoração
- Print do painel Problems mostrando os problemas (idealmente zerados) DEPOIS da refatoração

Esses dois prints precisam ser capturados rodando o VS Code com as extensões instaladas (Checkstyle, SpotBugs, Semgrep, SonarLint) — não é possível gerá-los sem executar o ambiente real localmente.